Active Object

son: 5 · Concurrency · Category: Concurrency · Level: Intermediate · Build with Wa

Today's Hook

- Today: Active Object as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Active Object structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Active Object
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/concurrency/activeobject/ActiveObject.java
package com.buildwithwaqar.patterns.concurrency.activeobject;

import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

/** Active Object — all calls run on a dedicated async servant thread. */
public final class ActiveObject implements AutoCloseable {
    private final ExecutorService servant = Executors.newSingleThreadExecutor(r -> {
        var t = new Thread(r, "active-object");
        t.setDaemon(true);
        return t;
    });
    private int value;

    public Future<Integer> add(int delta) {
        return servant.submit(() -> { value += delta; return value; });
    }

    public Future<Integer> get() {
        return servant.submit((Callable<Integer>) () -> value);
    }
```

Implementation (2)

```
    @Override public void close() { servant.shutdownNow(); }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.concurrency.activeobject;
public final class Demo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Active Object walkthrough ===");
    try (var ao = new ActiveObject()) {
      System.out.println("add2=" + ao.add(2).get());
      System.out.println("add3=" + ao.add(3).get());
      System.out.println("get=" + ao.get().get());
    }
  }
}
```

Walkthrough Output

```
=== Active Object walkthrough ===
add2=2
add3=5
get=5
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Active Object
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Active Object → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.